Network Properties

Java Properties

java.net.preferIPv4Stack (default: false)

If IPv6 is available on the operating system the underlying native socket will be an IPv6 socket. This allows Java(tm) applications to connect too, and accept connections from, both IPv4 and IPv6 hosts.

If an application has a preference to only use IPv4 sockets then this property can be set to true. The implication is that the application will not be able to communicate with IPv6 hosts.

java.net.preferIPv6Addresses (default: false)

If IPv6 is available on the operating system the default preference is to prefer an IPv4-mapped address over an IPv6 address. This is for backward compatibility reasons - for example applications that depend on access to an IPv4 only service or applications that depend on the %d.%d.%d.%d representation of an IP address. This property can be set to try to change the preferences to use IPv6 addresses over IPv4 addresses. This allows applications to be tested and deployed in environments where the application is expected to connect to IPv6 services.

networkaddress.cache.ttl

Specified in java.security to indicate the caching policy for successful name lookups from the name service.. The value is specified as as integer to indicate the number of seconds to cache the successful lookup.

A value of -1 indicates "cache forever". The default behavior is to cache forever when a security manager is installed, and to cache for an implementation specific period of time, when a security manager is not installed.

networkaddress.cache.negative.ttl (default: 10)

Specified in java.security to indicate the caching policy for un-successful name lookups from the name service.. The value is specified as as integer to indicate the number of seconds to cache the failure for un-successful lookups.

A value of 0 indicates "never cache". A value of -1 indicates "cache forever".

http.proxyHost (default: <none>)

http.proxyPort (default: 80 if http.proxyHost specified)

http.nonProxyHosts (default: <none>

ftp.proxyHost (default: <none>)

ftp.proxyPort (default: 80 if ftp.proxyHost specified)

ftp.nonProxyHosts (default: <none>)

http.proxyHost and http.proxyPort indicate the proxy server and port that the http protocol handler will use.

http.nonProxyHosts indicates the hosts which should be connected too directly and not through the proxy server. The value can be a list of hosts, each seperated by a |, and in addition a wildcard character (*) can be used for matching. For example: -Dhttp.nonProxyHosts="*.foo.com|localhost".

ftp.proxyHost and ftp.proxyPort indicate the proxy server and port that the ftp protocol handler will use. ftp.nonProxyHosts is similiar to http.nonProxyHosts and indicates the hosts that should be connected too directly and not through the proxy server.

http.agent (default: Java1.4.0)

Indicates the User-Agent request header sent in http requests.

http.auth.digest.validateServer (default: false)

http.auth.digest.validateProxy (default: false)

http.auth.digest.cnonceRepeat (default: 5)

These system properties modify the behavior of the HTTP digest authentication mechanism. Digest authentication provides a limited ability for the server to authenticate itself to the client (ie. by proving that it knows the users password). However, not all servers support this capability and by default the check is switched off. The first two properties above can be set to true, to enforce this check, for either authentication with an origin, or a proxy server respectively.

It is not normally necessary to set the third property (http.auth.digest.cnonceRepeat). This determines how many times a cnonce value is reused. This can be useful when the MD5-sess algorithm is being used. Increasing the value reduces the computational overhead on both the client and the server by reducing the amount of material that has to be hashed for each HTTP request.

http.auth.ntlm.domain:

Similar to other HTTP authentication schemes, NTLM uses the java.net.Authenticator class to acquire usernames and passwords when they are needed. However, NTLM also needs the NT domain name. There are three options for specifying the domain:

1. Do not specify it. In some environments, the domain is not actually required and the application need not specify it.
2. The domain name can be encoded within the username by prefixing the domain name followed by a back-slash '\' before the username. With this method, existing applications that use the Authenticator class do not need to be modified, so long as users are made aware that this notation must be used.
3. If a domain name is not specified as in method 2) and the system property "http.auth.ntlm.domain" is defined, then the value of this property will be used as the domain name.

http.keepAlive (default: true)

Indicates if keep alive (persistent) connections should be supported. Persistent connections improve performance by allowing the underlying socket connection be reused for multiple http requests.

The default is true and thus persistent connections will be used with http 1.1 servers. Set to 'false' to disable the use of persistent connections.

http.maxConnections (default: 5)

If HTTP keep-alive is enabled, this value is the number of idle connections that will be simultaneously kept alive, per-destination.

SOCKS protocol support settings

The SOCKS username and password are acquired in the following way. First, if the application has registered a java.net.Authenticator default instance, then this will be queried with the protocol set to the string "SOCKS5", and the prompt set to to the string "SOCKS authentication". If the authenticator does not return a username/password or if no authenticator is registered then the system checks for the user preferences "java.net.socks.username" and "java.net.socks.password". If these preferences do not exist, then the system property "user.name" is checked for a username. In this case, no password is supplied.

socksProxyHost

socksProxyPort (default: 1080)

Indicates the name of the SOCKS proxy server and the port number that will be used by the SOCKS protocol layer. If socksProxyHost is specified then all TCP

sockets will use the SOCKS proxy server to establish a connection or accept one. The SOCKS proxy server can either be a SOCKS v4 or v5 server and it has to allow for unauthenticated connections.

Sun implementation-specific properties

These properties may not be supported in future releases.

sun.net.inetaddr.ttl

This is a sun private system property which corresponds to networkaddress.cache.ttl. It takes the same value and has the same meaning, but can be set as a command-line option. However, the preferred way is to use the security property mentioned above.

sun.net.inetaddr.negative.ttl

This is a sun private system property which corresponds to networkaddress.cache.negative.ttl. It takes the same value and has the same meaning, but can be set as a command-line option. However, the preferred way is to use the security property mentioned above.

sun.net.client.defaultConnectTimeout (default: -1)

sun.net.client.defaultReadTimeout (default: -1)

These properties specify the default connect and read timeout (resp.) for the protocol handler used by java.net.URLConnection.

sun.net.client.defaultConnectTimeout specifies the timeout (in milliseconds) to establish the connection to the host. For example for http connections it is the timeout when establishing the connection to the http server. For ftp connection it is the timeout when establishing the connection to ftp servers.

sun.net.client.defaultReadTimeout specifies the timeout (in milliseconds) when reading from input stream when a connection is established to a resource.

sun.net.http.retryPost (default: true)

It determines if an unsuccessful HTTP POST request will be automatically resent to the server. Unsuccessful in this case means the server did not send a valid HTTP response or an IOException occurred.

JNDI DNS service provider settings

These properties may not be supported in future releases.

sun.net.spi.nameservice.provider.<n>=<default|dns,sun|...>

Specifies the name service provider that you can use. By default, Java will use the system configured name lookup mechanism, such as file, nis, etc. You can specify your own by setting this option. <n> takes the value of a positive number, it indicates the precedence order with a small number takes higher precendence over a bigger number. In J2SE™ 1.4, we have provided one DNS name service provider through JNDI, which is called "dns,sun".

sun.net.spi.nameservice.nameservers=<server1_ipaddr,server2_ipaddr ...>

You can specify a comma separated list of IP addresses that point to the DNS servers you want to use. If the sun.net.spi.nameservice.nameservers property is not defined, then the provider will use any name servers already configured in the platform DNS configuration.

sun.net.spi.nameservice.domain=<domainname>

This property specifies the default DNS domain name, for instance, eng.sun.com. If the sun.net.spi.nameservice.domain property is not defined then the provider will use any domain or domain search list configured in the platform DNS configuration.